

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	G.Mohammed Shariff	Reg. No.:	192425378
Date:	19/08/2026	Duration:	60 minutes

Code of Conduct

I Mohammedshariif?192425378 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:mohammedshariff

Annexure A – Git & Docker Technical Assignment

TITLE:AI-Based Learning Management System
with Personalized Learning Recommendations
DOMAIN:EDUCATION

1. Problem Analysis

1.1 Problem Statement

Traditional Learning Management Systems (LMS) mainly provide common learning materials and courses to students without sufficiently considering their individual learning abilities, interests, academic performance, and learning progress. As a result, students may receive learning content that is either too easy, too difficult, or not relevant to their current learning needs. This can reduce learning efficiency and make it difficult for students to identify which course or topic they should study next.

1.2 Project Objectives

1. To develop a web-based Learning Management System for students.
2. To provide secure user registration and authentication.
3. To allow students to access and manage available learning courses.
4. To track student learning progress and course completion.
5. To conduct assessments and record student performance.
6. To analyze student learning behavior and academic performance.
7. To provide personalized course and learning-resource recommendations using AI-based techniques.

1.3 Functional Requirements

S. No.	Functional Requirement	Description
1	User Registration	Students can create an account in the system.
2	User Login	Registered users can securely log in to the application.
3	Student Dashboard	Students can view their courses, progress, scores, and recommendations.
4	Course Management	Students can browse and access available courses.
5	Learning Progress	The system records completed courses, lessons, and learning activities.
6	Assessment	Students can attend quizzes or assessments and receive scores.
7	Performance Analysis	The system analyzes assessment scores and learning progress.
8	Personalized Recommendation	The system recommends suitable courses or learning resources based on student data.
9	Database Management	User, course, assessment, and progress information is stored in a database.
10	API Communication	The frontend communicates with the backend through APIs.

1.4 Non-Functional Requirements

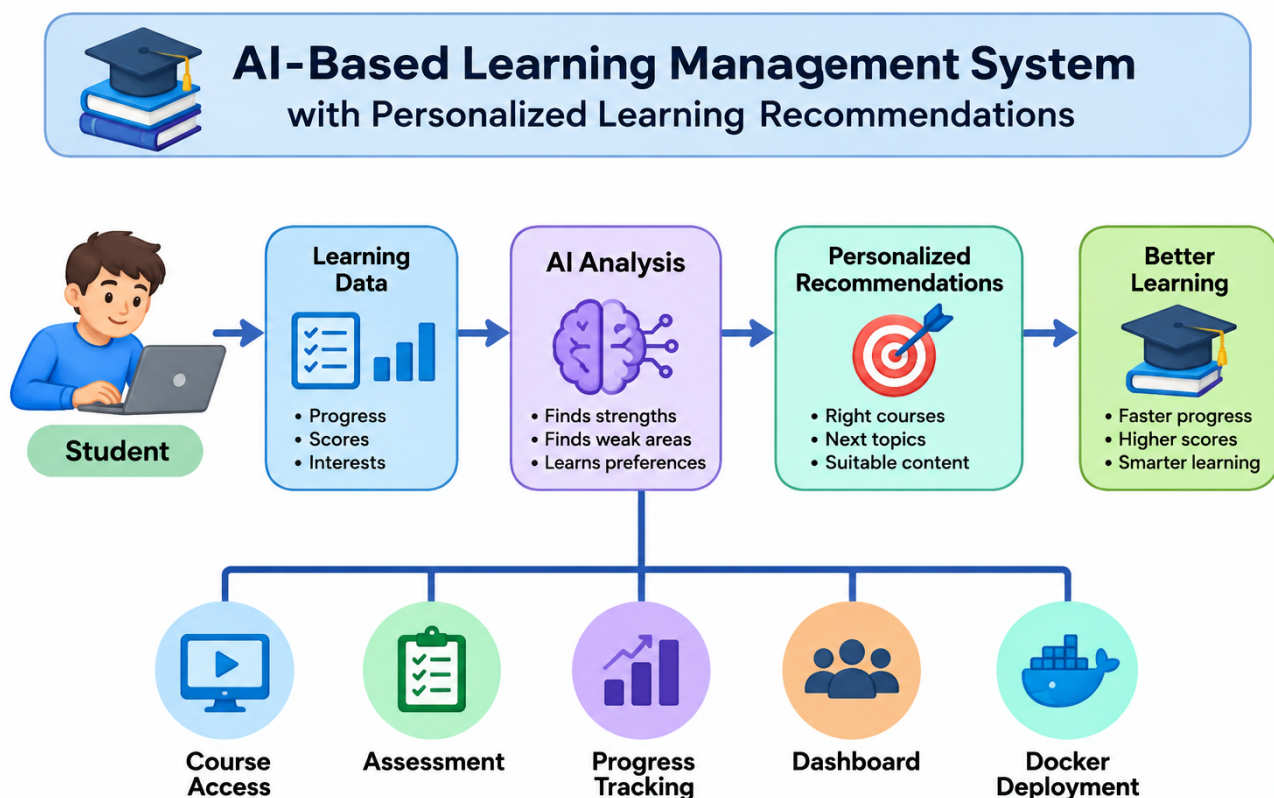
The system should satisfy the following non-functional requirements:

- **Performance:** The application should respond to user requests efficiently.
- **Security:** User credentials and application data should be protected.
- **Reliability:** The system should operate consistently without unnecessary failures.
- **Scalability:** The application should support additional users, courses, and features in the future.
- **Maintainability:** The source code should be organized into manageable modules.
- **Portability:** Docker should allow the application to run consistently across different environments.
- **Usability:** The interface should be simple and easy for students to navigate.

1.5 Expected Outcomes

The expected outcome is a functional **AI-Based Learning Management System** that provides students with a centralized platform for accessing courses, completing assessments, monitoring their learning progress, and receiving personalized learning recommendations.

The project is also expected to demonstrate practical implementation of **Git-based version control** and **Docker-based containerization**.

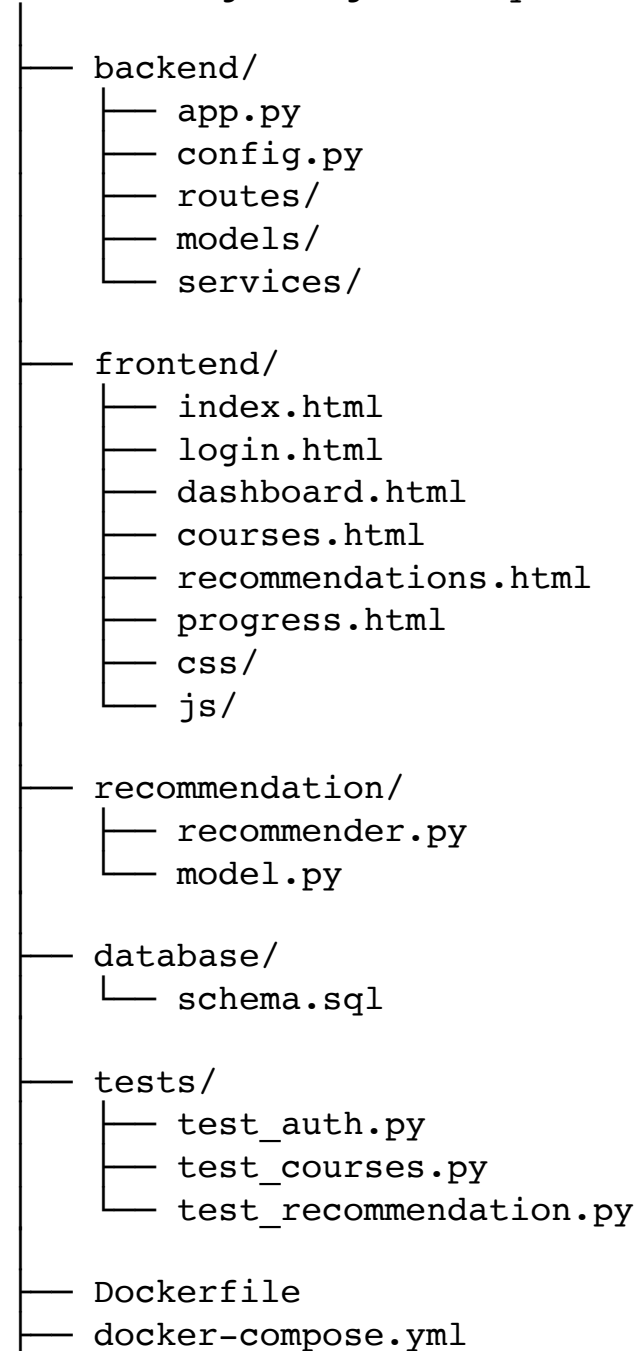


2. Project Structure Design

2.1 Project Repository Structure

The proposed **AI-Based Learning Management System with Personalized Learning Recommendations** is organized into separate modules for the frontend, backend, database, recommendation engine, testing, and deployment. This modular structure improves maintainability, scalability, and collaboration among developers.

AI-Learning-Management-System/



- requirements.txt
- .gitignore
- .dockerignore
- README.md
- .env

2.2 Description of Major Folders and Files

Folder/File	Purpose
backend/	Contains the server-side application and REST API implementation.
app.py	Main entry point of the Flask backend application.
config.py	Stores application and database configuration.
routes/	Contains API routes for authentication, courses, progress, and recommendations.
models/	Contains database models representing users, courses, assessments, and recommendations.
services/	Contains backend business logic and supporting services.
frontend/	Contains the user interface of the LMS.
index.html	Main landing page of the application.
login.html	Provides the user login interface.
dashboard.html	Displays student information, progress, scores, and recommendations.
courses.html	Displays available courses and learning materials.
recommendations.	Displays personalized course and topic recommendations.
progress.html	Displays student learning progress and performance.
css/	Contains CSS files used for interface styling.
js/	Contains JavaScript files for frontend functionality and API communication.
recommendation/	Contains the AI-based recommendation functionality.
recommender.py	Processes student information and generates personalized recommendations.
model.py	Contains the recommendation model or algorithm used by the system.
database/	Contains database-related files.
schema.sql	Defines tables and database structure required by the application.
tests/	Contains automated test cases for validating application functionality.
test_auth.py	Tests authentication functionality.
test_courses.py	Tests course-related functionality.

<code>test_recommendation.py</code>	Tests the recommendation system.
<code>Dockerfile</code>	Defines the instructions required to build the application Docker image.
<code>docker-compose.yml</code>	Defines and manages application services, containers, networks, and volumes.
<code>requirements.txt</code>	Lists Python libraries and dependencies required by the backend.
<code>.gitignore</code>	Specifies files and folders that should not be tracked by Git.
<code>.dockerignore</code>	Specifies files that should not be copied into the Docker image.
<code>README.md</code>	Provides project information, installation instructions, usage instructions, and deployment details.
<code>.env</code>	Stores environment-specific configuration values such as database credentials and secrets.

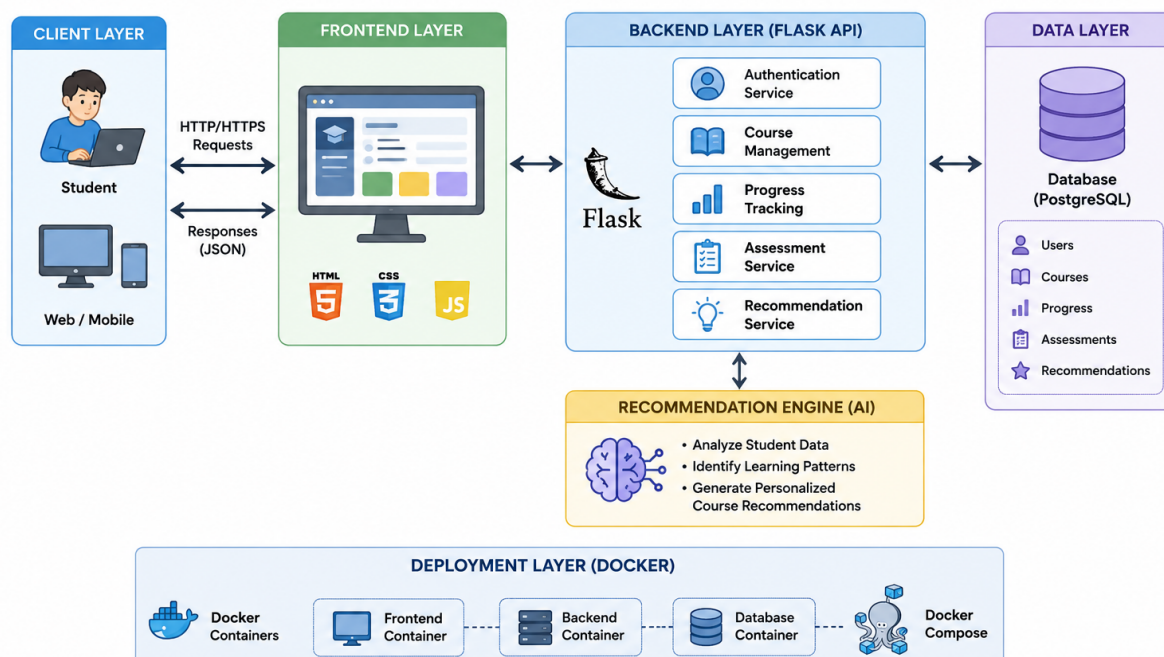
2.3 Justification of the Project Structure

The repository follows a **modular architecture** in which each major responsibility is separated into its own directory. The `frontend` directory handles the user interface, while the `backend` directory manages server-side operations and APIs. The `recommendation` directory separates the AI-based recommendation logic from the main application, making it easier to modify or improve the recommendation algorithm in the future.

This structure provides several advantages:

- **Modularity:** Each component has a specific responsibility.
- **Maintainability:** Individual modules can be modified without affecting the entire application.
- **Scalability:** New modules and features can be added easily.
- **Collaboration:** Multiple developers can work on different modules using Git branches.

AI-Based Learning Management System System Architecture



3. Git Repository Initialization

3.1 Objective

The Git repository is created to maintain the source code of the **AI-Based Learning Management System with Personalized Learning Recommendations**. Git provides version control, allowing developers to track changes, create branches, collaborate, restore previous versions, and maintain a complete history of the project.

3.2 Step-by-Step Git Repository Initialization

Step 1: Create the Project Folder

```
mkdir AI-Learning-Management-System
```

```
cd AI-Learning-Management-System
```

Purpose: Creates the main project directory and moves into it.

Step 2: Create the Initial Project Structure

```
mkdir backend frontend database recommendation tests
touch README.md requirements.txt Dockerfile docker-
compose.yml .gitignore .dockerignore
```

The project will now have:

AI-Learning-Management-System/

```
|— backend/
|— frontend/
|— database/
|— recommendation/
|— tests/
|
|— Dockerfile
|— docker-compose.yml
|— requirements.txt
|— .gitignore
|— .dockerignore
|— README.md
```

Step 3: Initialize Git

Run:

```
git init
```

Expected output:

```
Initialized empty Git repository in .../AI-Learning-Management-System/.git/
```

Purpose:

The `git init` command converts the project directory into a Git repository. It creates a hidden `.git` directory that stores Git's internal information, including commit history, branches, configuration, and tracked file information.

Step 4: Check Repository Status

```
git status
```

This displays the files that Git currently considers **untracked** or modified.

Example:

Untracked files:

```
Dockerfile
README.md
backend/
database/
frontend/
recommendation/
tests/
```

Step 5: Configure Git User Information

If Git has not been configured previously:

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

Check the configuration:

```
git config --global --list
```

This identifies the author of commits.

Step 6: Add Files to the Staging Area


```
git add .
```

The `.` means all eligible files and directories in the current project are added to the staging area.

Check:

```
git status
```

The files should now appear under **Changes to be committed**.

Step 7: Create the Initial Commit

```
git commit -m "chore: initialize project"
```

A commit creates a permanent snapshot of the staged project files.

Check the commit:

```
git log --oneline
```

Example:

```
a81f32c chore: initialize project
```

3.3 Create the Main Branch

Modern Git repositories normally use `main` as the primary branch.

```
git branch -M main
```

Check the current branch:

```
git branch
```

Expected:

```
* main
```

The `*` indicates the currently active branch.

3.4 Connect the Repository to GitHub

After creating an empty repository on GitHub, connect the local repository:

```
git remote add origin https://github.com/USERNAME/AI-Learning-Management-System.git
```

Verify:

```
git remote -v
```

Then push the project:

```
git push -u origin main
```

The `-u` option establishes `origin/main` as the upstream branch, making future pushes simpler.

Replace `USERNAME` with your actual GitHub username. Do not use the example URL as your real repository address.

3.5 Essential Git Files and Directories

File / Directory	Purpose
<code>.git/</code>	Internal Git directory containing repository metadata, objects, references, configuration, and commit information.
<code>.gitignore</code>	Specifies files that Git should not track, such as <code>.env</code> , virtual environments, and build artifacts.
<code>README.md</code>	Provides project description, installation instructions, usage information, and documentation.
<code>requirements</code>	Lists Python dependencies required to run the application.
<code>Dockerfile</code>	Contains instructions for creating the application's Docker image.
<code>docker-compose.yml</code>	Defines application services, containers, networks, volumes, and environment configuration.
<code>.dockerignore</code>	Prevents unnecessary files from being copied into the Docker build context.

3.6 Recommended `.gitignore`

For this project, use:

```
__pycache__/  
*.pyc  
.venv/  
venv/  
.env  
.DS_Store  
.pytest_cache/  
.vscode/  
.idea/  
*.log
```

Why?

For example, `.env` may contain database passwords or secret keys. It should **not** be committed to a public GitHub repository.

3.7 Recommended `.dockerignore`

```
.git/  
.github/  
.gitignore  
.dockerignore  
.venv/  
venv/  
__pycache__/  
*.pyc  
.env  
.pytest_cache/  
.vscode/  
.idea/  
*.log
```

This prevents unnecessary files from being sent to Docker during image building.

3.8 Git Initialization Workflow

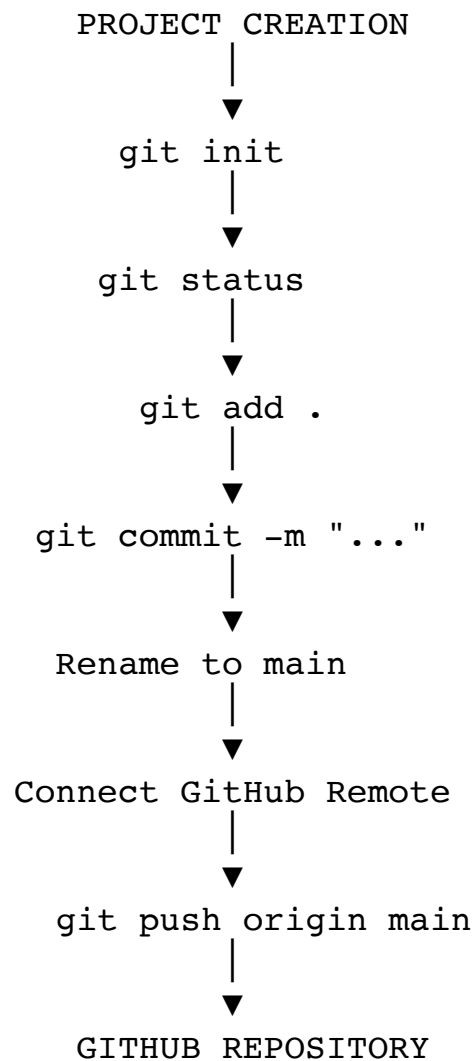


Figure 3: Git Repository Initialization Workflow

3.9 Evidence/Screenshots Required

Screenshot	Command / Evidence
Screenshot 1	<code>git init</code> successful
Screenshot 2	<code>git status</code> showing project files
Screenshot 3	<code>git log --oneline</code> showing initial commit
Screenshot 4	GitHub repository containing project files

4. Git Branching Strategy

4.1 Selected Branching Model

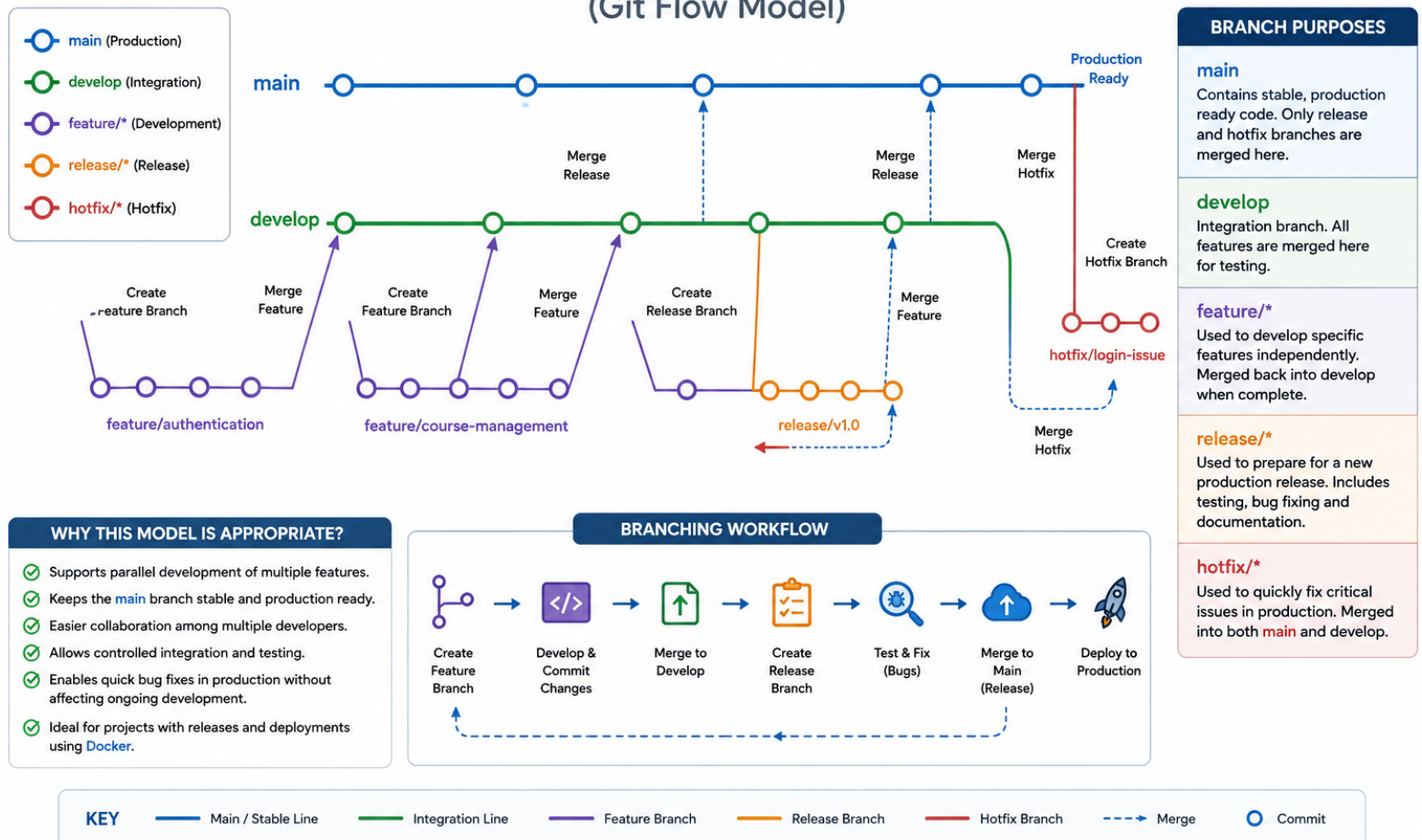
For the **AI-Based Learning Management System with Personalized Learning Recommendations**, a **Git Flow-based branching strategy** is selected.

The strategy uses five types of branches:

- `main`
- `develop`
- `feature/*`
- `release/*`
- `hotfix/*`

This model is suitable because the project contains multiple independent modules such as authentication, course management, progress tracking, assessment, AI recommendations, and Docker deployment.

4.2 Branching Structure



4.3 Branch Description

Branch	Purpose
main	Contains the stable and production-ready version of the application.
develop	Integration branch where completed features are combined and tested.
feature/ *	Used to develop individual project features independently.
release/ *	Used to prepare and test a stable version before merging into main .
hotfix/*	Used to immediately fix critical problems in the production version.

4.4 Feature Branches for This Project

```
feature/authentication  
feature/course-management  
feature/student-dashboard  
feature/progress-tracking  
feature/assessment  
feature/recommendation-engine  
feature/docker  
feature/testing
```

For example:

```
git checkout develop
```

```
git checkout -b feature/authentication
```

After completing the authentication module:

```
git add .  
git commit -m "feat: implement user authentication"
```

Then merge it into develop:

```
git checkout develop  
git merge feature/authentication
```

4.5 Release Branch

```
git checkout develop  
git checkout -b release/v1.0
```

The release branch is used for:

- Final testing
- Bug fixing
- Documentation
- Docker testing
- Final verification

After successful testing, it can be merged into `main`.

```
git checkout main
```

```
git merge release/v1.0
```

4.6 Hotfix Branch

If a critical error is discovered after deployment:

```
git checkout main  
git checkout -b hotfix/login-error
```

After fixing the problem:

```
git add .  
git commit -m "fix: resolve login authentication error"  
  
git checkout main  
git merge hotfix/login-error
```

This allows critical problems to be fixed without disrupting ongoing development.

4.7 Why This Branching Model Is Appropriate

The Git Flow-based model is appropriate for this project for the following reasons:

1. Independent Development

Each feature can be developed in its own branch without affecting the stable application.

2. Better Collaboration

Different developers can work simultaneously on authentication, courses, AI recommendations, frontend, and Docker.

3. Stable Main Branch

The `main` branch contains only tested and stable versions of the application.

4. Controlled Integration

The `develop` branch acts as an integration point where features are combined and tested.

5. Organized Releases

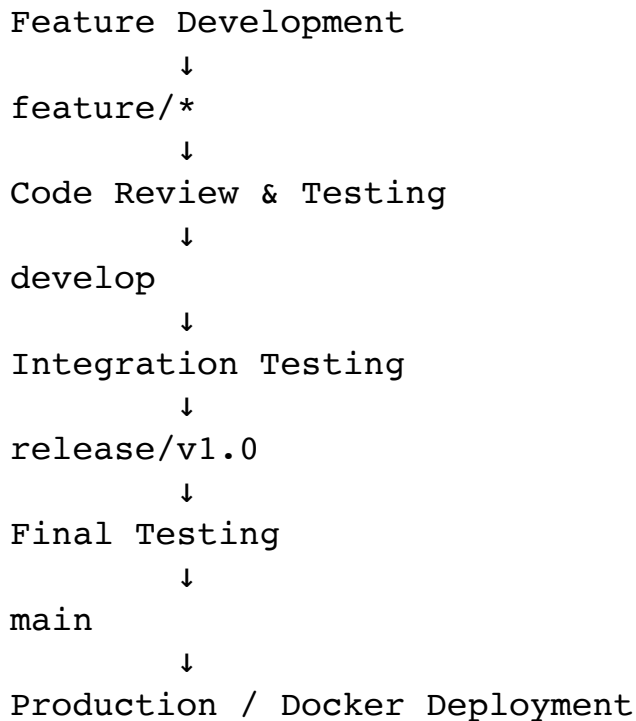
The `release` branch provides a controlled environment for final testing before deployment.

6. Quick Bug Fixes

The `hotfix` branch allows critical production issues to be resolved quickly.

4.8 Branching Workflow

The complete workflow is:



Overall justification: This branching strategy provides **separation of development work, controlled integration, stable releases, and emergency bug fixing**, making it appropriate for a full-stack AI-based LMS developed using Git and Docker.

5. Version Control Workflow

The Git workflow for the **AI-Based Learning Management System with Personalized Learning Recommendations** is designed to maintain an organized development process. The workflow includes feature branch creation, meaningful commits, merging, conflict resolution, and synchronization with the remote GitHub repository.

5.1 Overall Git Workflow

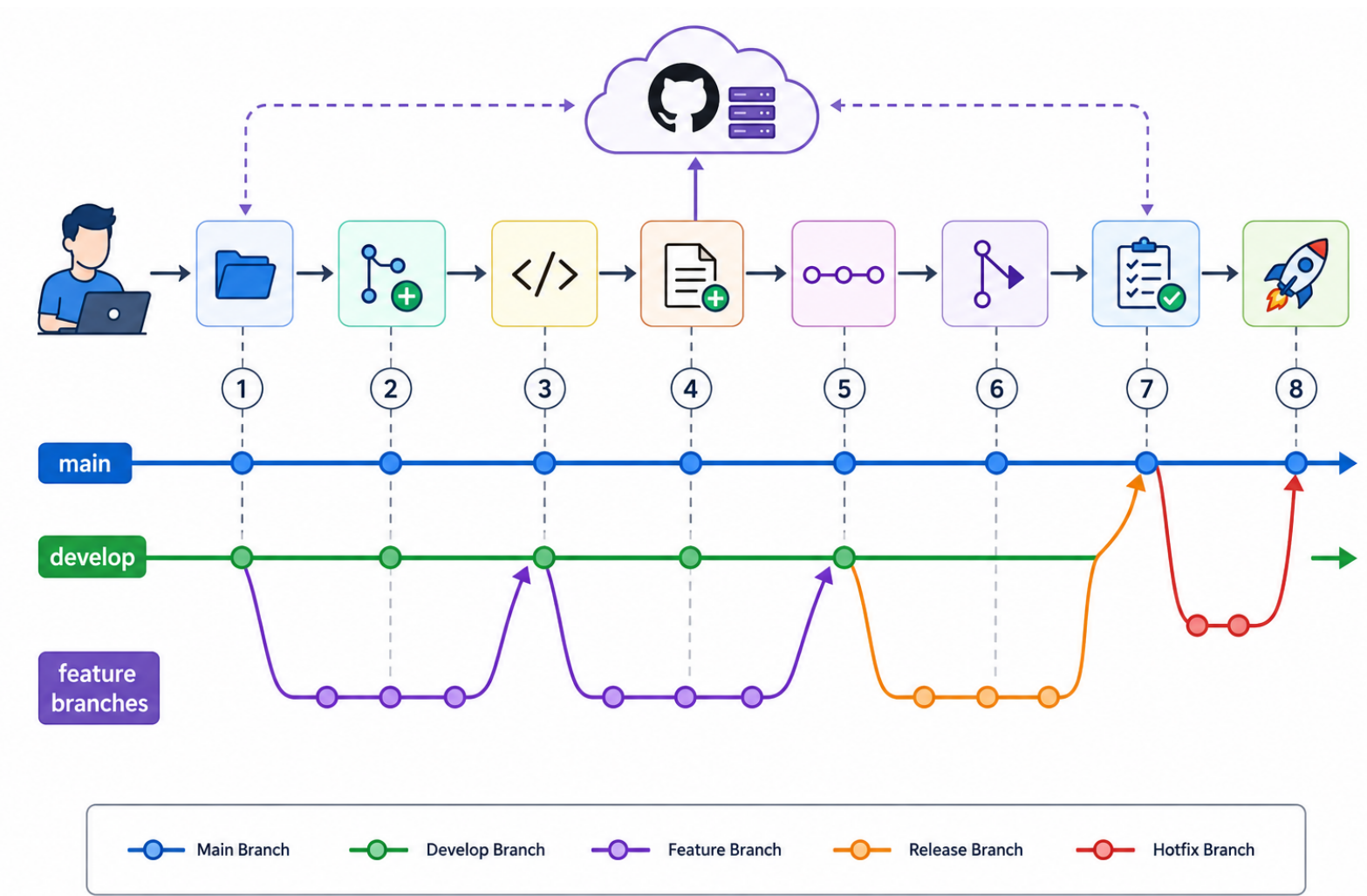


Figure 6: Version Control Workflow of the AI-Based Learning Management System

5.2 Step 1 – Check the Repository Status

The current state of the working directory is checked before starting development.

```
git status
```

Purpose:

`git status` displays modified, untracked, and staged files. It helps the developer understand the current state of the repository before performing another Git operation.

5.3 Step 2 – Switch to the Development Branch

```
git checkout develop
```

Purpose:

The `develop` branch is used for integrating completed features before they are released to the stable `main` branch.

5.4 Step 3 – Create a Feature Branch

A separate branch is created for each major feature.

For example, to develop the authentication module:

```
git checkout -b feature/authentication
```

Other possible branches are:

```
feature/course-management
```

```
feature/student-dashboard
```

```
feature/progress-tracking
```

```
feature/assessment
```

```
feature/recommendation-engine
```

```
feature/docker
```

Purpose:

Feature branches isolate individual development tasks from the main application. This allows features to be developed and tested independently without directly affecting the stable code.

5.5 Step 4 – Make Meaningful Changes

The developer implements the authentication functionality in the project.

After making changes, the modified files are checked:

```
git status
```

The files are then added to the staging area:

```
git add .
```

Purpose:

`git add` moves the selected changes into the staging area. The staging area allows the developer to select exactly which changes will be included in the next commit.

5.6 Step 5 – Create a Meaningful Commit

```
git commit -m "feat: implement user authentication"
```

Purpose:

A commit creates a permanent snapshot of the staged changes. A meaningful commit message makes it easier to understand the project's development history.

Examples of meaningful commits for this project include:

```
feat: implement user authentication
feat: add course management module
feat: implement student dashboard
feat: add learning progress tracking
feat: implement assessment module
feat: add personalized recommendation engine
docker: containerize backend application
test: add recommendation system tests
docs: update project documentation
fix: resolve login validation error
```

5.7 Step 6 – Merge the Feature Branch

After completing and testing the feature, switch back to the development branch:

```
git checkout develop
```

Merge the completed feature:

```
git merge feature/authentication
```

Purpose:

`git merge` combines the completed feature into the `develop` branch. This allows the feature to become part of the integrated application.

5.8 Step 7 – Conflict Resolution

A merge conflict can occur when two branches modify the same part of a file differently.

Check the repository:

```
git status
```

Git identifies the conflicting files.

A conflict may appear in a file as:

```
<<<<<< HEAD
```

```
Current develop branch code
```

```
=====
```

```
Feature branch code
```

```
>>>>>> feature/authentication
```

The developer manually selects or combines the correct code and removes the conflict markers.

After resolving the conflict:

```
git add .
```

```
git commit -m "fix: resolve merge conflict"
```

5.9 Step 8 – View Commit History

The complete development history can be displayed using:

```
git log --oneline --graph --all
```

Example:

```
* docs: update README
* docker: add Docker Compose configuration
* feat: add recommendation engine
* feat: add student dashboard
* fix: resolve merge conflict
* feat: implement authentication
* chore: initialize project
```

5.10 Step 9 – Synchronize with GitHub

Before pushing new changes, the local repository can be synchronized with the remote repository:

```
git pull origin develop
```

5.11 Step 10 – Release the Stable Version

After all features have been integrated and tested:

```
git checkout develop
git checkout -b release/v1.0
```

After final testing:

```
git checkout main
git merge release/v1.0
```

The stable version is then synchronized with GitHub:

```
git push origin main
```

Purpose:

The release branch provides a controlled environment for final testing before the application is merged into the production-ready `main` branch.

5.12 Git Operations and Their Purpose

Git Operation	Purpose
<code>git status</code>	Checks the current repository state.
<code>git checkout develop</code>	Switches to the development branch.
<code>git checkout -b feature/...</code>	Creates and switches to a new feature branch.
<code>git add .</code>	Adds modified files to the staging area.
<code>git commit</code>	Creates a permanent snapshot of changes.
<code>git merge</code>	Combines changes from one branch into another.
<code>git status</code> during conflict	Identifies files with merge conflicts.
Manual conflict resolution	Combines the correct code from different branches.
<code>git log</code>	Displays commit history.
<code>git pull</code>	Downloads and integrates remote changes.
<code>git push</code>	Uploads local commits to GitHub.
<code>git checkout main</code>	Switches to the stable production branch.

5.13 Complete Workflow Used in the Project

The complete workflow can be summarized as:

Create Feature



Create Feature Branch



Develop Feature



git add



Meaningful Commit



Test Feature



Merge into develop



Resolve Conflicts if Required



Integration Testing



Create Release Branch



Final Testing



Merge into main



git push



GitHub Repository

This workflow provides controlled development, clear version history, isolated feature development, reliable integration, and synchronization between the local project and the remote GitHub repository.

6. Commit History Analysis

6.1 Project Commit History

Show the commit history using:

```
git log --oneline --graph --all
```

Example:

```
* docs: update project documentation
* docker: add Docker Compose configuration
* docker: add application Dockerfile
* test: add recommendation system tests
* feat: implement personalized recommendation engine
* feat: add student progress tracking
* feat: implement assessment module
* feat: add student dashboard
* feat: implement course management
* feat: implement user authentication
* chore: initialize project
```

Figure 7: Git Commit History



Screenshot: Your actual `git log --oneline --graph --all`.

6.2 Commit Message Analysis

Use the following convention:

Prefix	Example	Purpose
feat:	feat: add student dashboard	New functionality
fix:	fix: resolve login error	Bug correction
test:	test: add API tests	Testing changes
docs:	docs: update README	Documentation

<code>docker:</code>	<code>docker: add Dockerfile</code>	Docker-related changes
<code>chore:</code>	<code>chore: initialize project</code>	Maintenance tasks

Why meaningful commits are important

Meaningful commit messages:

- Make project history easy to understand.
- Help developers identify when a feature was introduced.
- Make debugging easier.
- Allow changes to be traced.
- Help revert problematic changes.
- Improve collaboration between developers.
- Make project maintenance easier.

7. Docker Environment Design

7.1 Why Docker Is Suitable

Docker is suitable for the proposed LMS because the application contains multiple software dependencies such as Python, Flask, database libraries, and AI/recommendation libraries.

Docker packages the application and its dependencies into isolated containers. This ensures that the application behaves consistently across different development and deployment environments.

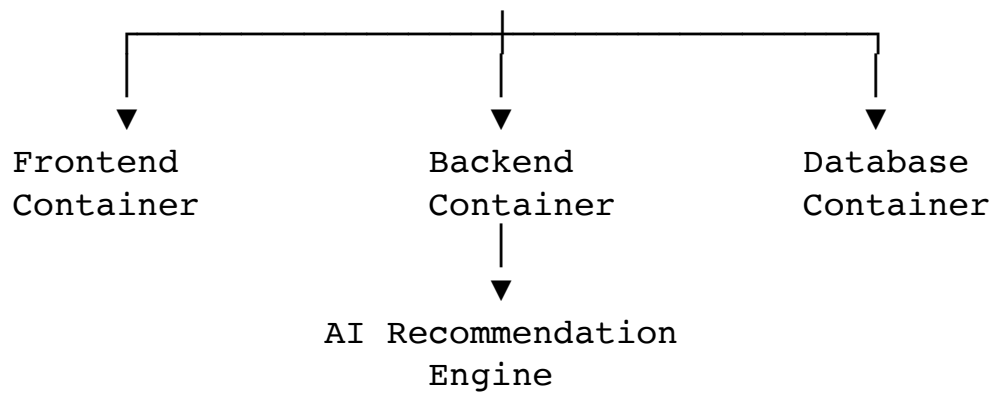
Main advantages

- Consistent environment
- Dependency isolation
- Portability
- Easy deployment
- Reproducibility
- Scalability
- Easier maintenance

7.2 Components Requiring Containerization

For this project, use three logical components:

DOCKER COMPOSE
|



Components

Component	Container	Purpose
Frontend	Yes	Provides LMS user interface
Flask Backend	Yes	Handles API and application logic
AI Recommendation Engine	Inside backend	Generates personalized recommendations
PostgreSQL	Yes	Stores application data

7.3 Docker Architecture

For the report, use this architecture:

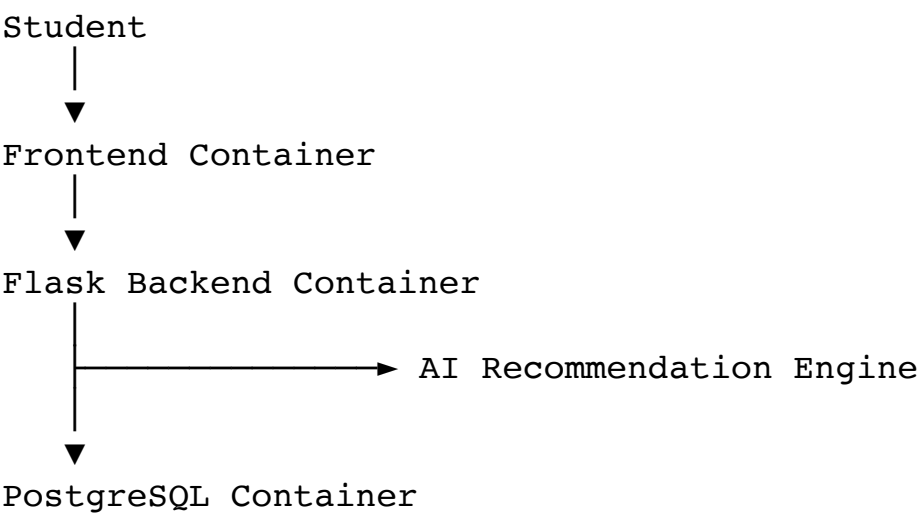


Figure 8: Docker Environment Architecture

8. Dockerfile Development

8.1 Dockerfile

Create a file named:

Dockerfile

Recommended Dockerfile:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

8.2 Explanation of Dockerfile Instructions

Instruction	Purpose
FROM	Selects the base Python image.
WORKDIR	Sets <code>/app</code> as the working directory inside the container.
COPY requirements.txt .	Copies the dependency file into the container.
RUN pip install	Installs required Python packages.
COPY . .	Copies the application source code into the container.
EXPOSE 5000	Indicates that the Flask application uses port 5000.

CMD	Starts the Flask application when the container runs.
-----	---

8.3 Docker Image Creation

Build the image:

```
docker build -t ai-lms .
```

Check the image:

```
docker images
```

Run the container:

```
docker run -p 5000:5000 ai-lms
```

Then access:

```
http://localhost:5000
```



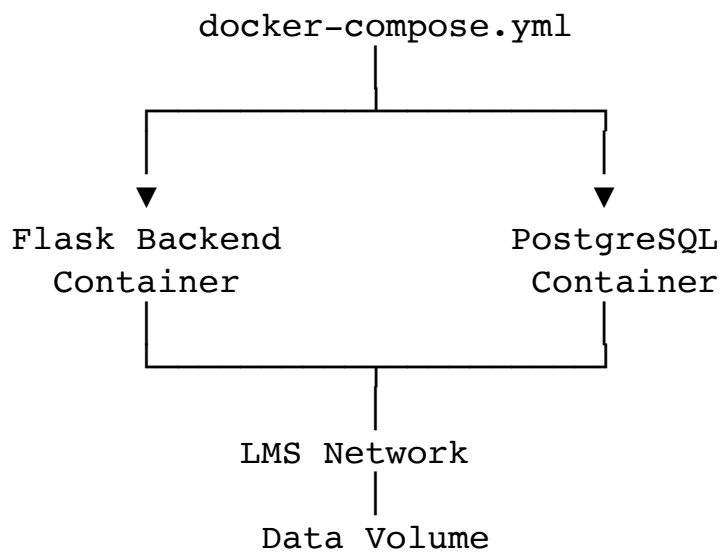
Screenshots:

1. `docker build`
2. `docker images`
3. `docker run`
4. LMS running in browser

9. Docker Compose Design

9.1 Docker Compose Architecture

Docker Compose is used to manage multiple services together.



9.2 Docker Compose Configuration

Create:

`docker-compose.yml`

Example:

```
services:

  web:
    build: .
    container_name: ai-lms-web
    ports:
      - "5000:5000"
    environment:
      DATABASE_HOST: db
      DATABASE_NAME: lms
      DATABASE_USER: lmsuser
      DATABASE_PASSWORD: lmspassword
    depends_on:
      - db

  db:
```

```
image: postgres:16
container_name: ai-lms-db
environment:
  POSTGRES_DB: lms
  POSTGRES_USER: lmsuser
  POSTGRES_PASSWORD: lmspassword
volumes:
  - postgres_data:/var/lib/postgresql/data
```

```
volumes:
  postgres_data:
```

```
networks:
  default:
    name: ai-lms-network
```

For the actual project, credentials should preferably be stored in an `.env` file rather than hard-coded in the Compose file.

9.3 Services

Web Service

web

Runs the Flask LMS application.

Database Service

db

Runs PostgreSQL and stores:

- Users
- Courses
- Assessments
- Scores
- Progress
- Recommendations

9.4 Networking

Docker Compose automatically creates a network allowing:

web container  db container

The backend connects to PostgreSQL using:

```
DATABASE_HOST=db
```

The service name `db` acts as the hostname inside the Docker network.

9.5 Volumes

```
volumes:
```

```
  - postgres_data:/var/lib/postgresql/data
```

The volume ensures that database information remains available even if the PostgreSQL container is stopped or recreated.

9.6 Environment Variables

Environment variables are used for configuration such as:

```
DATABASE_HOST
DATABASE_NAME
DATABASE_USER
DATABASE_PASSWORD
```

This separates configuration from application code and makes deployment easier.

9.7 Dependencies

```
depends_on:  
  - db
```

This indicates that the web service depends on the database service.

10. Container Deployment and Testing

10.1 Build Containers

Run:

```
docker compose build
```



Screenshot: Successful Docker build.

10.2 Start Application

```
docker compose up -d
```

Check running containers:

```
docker ps
```

Expected:

```
ai-lms-web  
ai-lms-db
```

 Screenshot: `docker ps`.

10.3 Check Container Logs

`docker compose logs`

Or:

`docker compose logs web`

This helps identify application errors.

10.4 Test the Application

Open:

`http://localhost:5000`

Test:

Login



Dashboard



Courses



Assessment



Progress



AI Recommendations

10.5 Testing Table

Test Case	Expected Result	Status
User Registration	User account created	Pass
User Login	Dashboard displayed	Pass
Course Access	Course displayed	Pass
Assessment	Score generated	Pass
Progress Tracking	Progress updated	Pass
Recommendation	Personalized courses displayed	Pass
Database Connection	Data stored successfully	Pass
Docker Deployment	Application starts successfully	Pass

10.6 Stop Containers

After testing:

```
docker compose down
```

If you want to stop containers while keeping database volumes:

```
docker compose stop
```

11. Benefits of Git and Docker

11.1 Git Benefits

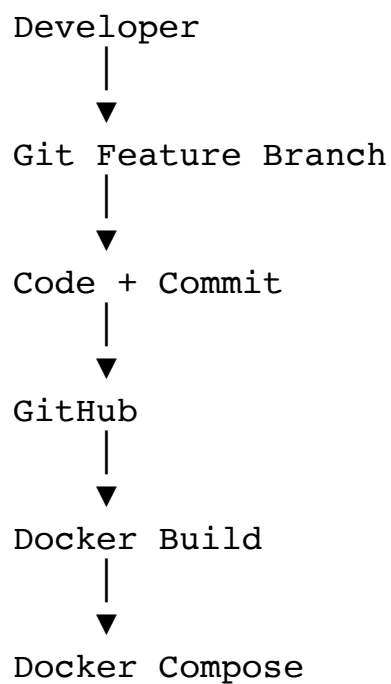
Benefit	Application to LMS
Version Control	Tracks every project modification
Collaboration	Multiple developers can work simultaneously

Branching	Each LMS feature can be developed independently
Rollback	Previous working versions can be restored
History	All development changes are recorded
Conflict Management	Conflicting changes can be identified and resolved

11.2 Docker Benefits

Benefit	Application to LMS
Portability	Application can run on different systems
Consistency	Same environment for development and deployment
Isolation	Dependencies remain separated
Deployment	Application can be started using Compose
Scalability	Services can be scaled independently
Maintainability	Environment configuration is standardized

11.3 Combined Git + Docker Workflow



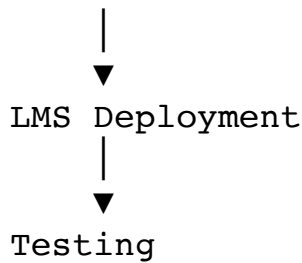


Figure 9: Integrated Git and Docker Development Workflow

12. Challenges and Improvements

12.1 Challenges Encountered

1. Git Merge Conflicts

Different feature branches may modify the same source files, resulting in merge conflicts.

Solution: Review conflicting code, resolve the differences, test the application, and commit the resolved version.

2. Dependency Management

Different Python libraries may have version compatibility issues.

Solution: Maintain dependencies in `requirements.txt` and use a fixed Docker base image.

3. Database Connectivity

The Flask application needs to communicate correctly with PostgreSQL inside Docker.

Solution: Use Docker Compose networking and the database service name as the database host.

4. Docker Configuration

Incorrect ports, environment variables, or service configurations can prevent containers from starting correctly.

Solution: Verify `docker-compose.yml`, inspect logs, and test services individually.

5. Recommendation Accuracy

The recommendation system may produce unsuitable recommendations if insufficient student data is available.

12.2 Future Enhancements

The system can be improved with:

AI Improvements

- Advanced recommendation algorithms
- Collaborative filtering
- Hybrid recommendation systems
- Student knowledge modeling
- Skill-gap detection
- Adaptive learning paths

LMS Improvements

- AI chatbot/tutor
- Personalized quizzes
- Real-time learning analytics
- Mobile application
- Notifications
- Instructor dashboard

DevOps Improvements

- GitHub Actions CI/CD
- Automated testing
- Docker image registry
- Cloud deployment
- Kubernetes orchestration
- Monitoring and logging

Final Report Structure

At this point, your complete report should be:

1. Problem Analysis

- Essential Git Files
- GitHub Synchronization

4. Git Branching Strategy

- Main
- Develop
- Feature
- Release

└─ Hotfix

5. Version Control Workflow

- └─ Branch Creation
- └─ Commits
- └─ Merge
- └─ Conflict Resolution
- └─ Synchronization

6. Commit History Analysis

- └─ Git Log
- └─ Commit Message Practices

7. Docker Environment Design

- └─ Docker Architecture
- └─ Containerized Components

8. Dockerfile Development

- └─ Dockerfile
- └─ Instruction Explanation

9. Docker Compose Design

- └─ Services
- └─ Networking
- └─ Volumes
- └─ Environment Variables
- └─ Dependencies

10. Container Deployment & Testing

- └─ Build
- └─ Run
- └─ Testing
- └─ Results

11. Benefits of Git and Docker

12. Challenges and Improvements

- └─ Challenges
- └─ Future Enhancements

13. Conclusion

14. References

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25